# ACKNOWLEDGMENT

We would like to express my sincere gratitude to our respected Principal and Head of the Department for providing the necessary institutional facilities, academic infrastructure, and encouragement that enabled the successful completion of this project.

We convey our deep sense of appreciation and heartfelt thanks to our project guide, **Ms. Nazera Anjum (Mam)**, for their invaluable guidance, constructive suggestions, and continuous support throughout the software development life cycle—from requirement analysis and system architecture design to implementation and testing.

We are also thankful to all faculty members, technical laboratory assistants, and staff members of the Computer/Information Technology Department for their indirect assistance and timely technical inputs.

Finally, I extend my appreciation to my parents, family members, and peers for their constant motivation, patience, and cooperation during the preparation and submission of this project report.

**Hanzala, Owais, Usaid, Krishnakant**

**Roll No:** 09, 14, 18, 30.

**Department of Computer Engineering**

**Maulana Mukthar Ahmad Nadvi Technical Campus**

# ABSTRACT

Weather conditions play a crucial role in daily planning, travel, agriculture, and transportation. Traditional methods of retrieving weather reports through television, newspapers, or multiple web searches are often time-consuming and may fail to deliver immediate, real-time meteorological data. The Weather Forecast System Using API is a responsive, web-based application designed to provide users with fast, centralized, and accurate weather updates for any global location.

Developed under the linear-sequential **Waterfall Software Development Life Cycle (SDLC)** model, the application bridges user inputs with external meteorological services. By entering a target city name, the system constructs and dispatches an HTTP request to an online **Weather API** (such as Open Weather Map). The API returns real-time data in JSON format, which the backend—built using **Python** and the **Flask** microframework—parses and processes to extract key parameters, including current temperature, humidity levels, wind speed, atmospheric pressure, and weather conditions.

These metrics are rendered dynamically on a clean frontend developed using **HTML, CSS, and JavaScript**, supported by comprehensive error handling for invalid queries and network dropouts. The system also incorporates an optional **SQLite** database schema for query history tracking. This project highlights practical API integration, real-time data handling, and standard software engineering practices, delivering a reliable, user-friendly tool for daily weather tracking.